

Portland State University
Maseeh College of Engineering and Computer Science
Department of Electrical and Computer Engineering

ECE 486/586: Computer Architecture

Term Project Report

MIPS-Lite Execution-Driven Pipeline Simulator

Spring 2026

Team members:

Hanisha Produtur	hanisha@pdx.edu
Venkat Sai Sumanth Koyada	koyada@pdx.edu
Nikhil Swarna	nikhilsw@pdx.edu
Venkata Sriram Kamarajugadda	vkamaraj@pdx.edu

Program: M.S. Electrical and Computer Engineering

Instructor: Prof. Yuchen Huang yuchen@pdx.edu

Teaching assistant: Lynn Nguyen diemhan@pdx.edu

Submission date: June 5, 2026

Submitted in partial fulfillment of the requirements for ECE 486/586, Spring 2026.

Contents

List of tables	iii
List of figures	iv
1 Introduction	1
1.1 Introduction	1
2 MIPS-Lite ISA and architecture	2
2.1 Introduction	2
2.2 Instruction set	2
2.2.1 Instruction categories	2
2.2.2 Instruction formats	2
2.3 5-stage pipeline architecture	2
2.4 Memory model	3
3 Design decisions and assumptions	4
3.1 Functional simulator design	4
3.2 Pipeline hazard handling	4
3.2.1 RAW data hazards – no forwarding	4
3.2.2 RAW data hazards – with forwarding	4
3.3 Branch handling	4
3.4 Assumptions	5
4 Design methodology	6
4.1 Introduction	6
4.2 Simulation environment	6
4.3 Memory image format	6
4.4 Instruction decoding	6
4.5 Hazard detection	7
4.6 Simulation results	7
5 Implementation of the MIPS-Lite simulator	8
5.1 Introduction	8
5.2 Trace reader module	8
5.3 Instruction decoder module	8
5.4 Functional simulator module	8

5.5	Pipeline timing simulator module	8
5.5.1	Stall insertion	8
5.5.2	Branch flush	9
5.5.3	Lockstep execution model	9
5.6	Statistics module	9
6	Simulation results and analysis	10
6.1	Instruction frequency breakdown	10
6.2	Final machine state	10
6.2.1	Program counter	10
6.2.2	General-purpose registers	10
6.2.3	Memory	11
6.3	Stall conditions and penalties	11
6.3.1	Pipeline timing model	11
6.3.2	Same-cycle read/write rule	12
6.3.3	No-forwarding stall rules	12
6.3.4	Forwarding stall rules	12
6.4	Pipeline diagram – first five instructions	12
6.5	Data hazard statistics	13
6.6	No-forwarding timing	13
6.7	Forwarding timing	14
6.8	Execution time and speedup	14
6.9	Complete results summary	15
6.10	Simulator output screenshots	15
7	Roles and responsibilities	19
	Conclusion	21
A	Source code	22
B	Sample memory image reference output	35
C	Demo-day trace – complete memory output	36

List of Tables

2.1	MIPS-Lite instruction formats	2
2.2	5-stage pipeline stages	3
3.1	RAW stall penalties without forwarding	4
3.2	RAW stall penalties with forwarding	4
6.1	Instruction frequency breakdown	10
6.2	Final program counter	10
6.3	Final register state (changed registers only)	11
6.4	Final memory state – selected entries (full table in Appendix B)	11
6.5	No-forwarding RAW hazard stall penalties	12
6.6	Forwarding RAW hazard stall penalties	12
6.7	Pipeline diagram – first 5 instructions (no stalls)	13
6.8	Data hazard statistics	13
6.9	No-forwarding timing results	13
6.10	Forwarding timing results	14
6.11	Execution time and forwarding speedup	14
6.12	Complete results summary	15
7.1	Project roles and responsibilities (part 1 of 2)	19
7.2	Project roles and responsibilities (part 2 of 2)	20

List of Figures

6.1	Simulator output – instruction counts and final register state	16
6.2	Simulator output – final memory state	17
6.3	Simulator output – timing results and speedup	18

1 Introduction

1.1 Introduction

Modern processor design is characterized by an ever-widening gap between processor speed and main memory latency. To bridge this gap, architects employ increasingly sophisticated pipeline and memory hierarchy techniques. This project addresses both dimensions by developing a full behavioral simulator for the MIPS-Lite instruction set architecture (ISA) running on a classical 5-stage in-order pipeline.

The MIPS-Lite ISA is a simplified subset of the MIPS architecture, supporting 18 distinct instructions across four categories: arithmetic, logical, memory access, and control flow. The simulator is execution-driven, taking a memory image file as input and producing detailed output covering functional execution results, pipeline timing, and performance metrics.

A key design goal is to implement and compare two versions of the timing simulator: one that models pipeline execution without any forwarding (data bypass) paths, and one that incorporates full forwarding to resolve data hazards wherever possible. This allows a direct quantitative comparison of the performance benefit of forwarding in terms of clock cycle reduction and speedup.

The simulator is implemented in C (C11 standard) and consists of six cooperating modules: trace reader, instruction decoder, functional simulator, timing simulator, statistics, and a top-level driver. All three simulation modes (functional only, no-forwarding timing, forwarding timing) run in a single program invocation.

2 MIPS-Lite ISA and architecture

2.1 Introduction

This chapter describes the MIPS-Lite instruction set architecture and the 5-stage pipeline model that forms the basis of the simulator. Understanding these foundations is essential before examining the implementation and results.

2.2 Instruction set

The MIPS-Lite ISA supports 18 instructions organized into four categories. All instructions are 32 bits wide and fall into one of two encoding formats.

2.2.1 Instruction categories

- Arithmetic instructions: ADD, ADDI, SUB, SUBI, MUL, MULI
- Logical instructions: OR, ORI, AND, ANDI, XOR, XORI
- Memory access instructions: LDW, STW
- Control flow instructions: BZ, BEQ, JR, HALT

All arithmetic computations treat operands as signed integers and use two's complement binary arithmetic. This applies to register contents, immediate values, effective address calculations, and branch target computations.

2.2.2 Instruction formats

Instructions are encoded in one of two formats.

Table 2.1: MIPS-Lite instruction formats

Format	Fields	Widths (bits)	Instructions
R-type	Opcode Rs Rt Rd Unused	6 5 5 5 11	ADD, SUB, MUL, OR, AND, XOR
I-type	Opcode Rs Rt Immediate	6 5 5 16	ADDI, SUBI, MULI, ORI, ANDI, XORI, LDW, STW, BZ, BEQ, JR, HALT

2.3 5-stage pipeline architecture

The simulator models a classical 5-stage in-order pipeline.

Table 2.2: 5-stage pipeline stages

Stage	Name	Description
IF	Instruction Fetch	Fetch instruction from memory at current PC
ID	Instruction Decode	Decode instruction, read registers, detect hazards
EX	Execute	ALU operations, branch condition evaluation, target computation
MEM	Memory Access	Load/store data from/to memory
WB	Write Back	Write result to destination register

2.4 Memory model

The simulator uses a flat 4 kB shared address space (1024 words) initialized from the memory image file. Each line in the image represents one 32-bit word in big-endian hexadecimal format, with addresses starting at 0 and incrementing by 4. The program counter and all general-purpose registers (R1–R31) are initialized to 0. Register R0 is hardwired to 0.

3 Design decisions and assumptions

3.1 Functional simulator design

The functional simulator maintains the full machine state: PC, 32 general-purpose registers, and 4 kB memory. On each instruction, the simulator decodes the opcode, reads operands, executes the operation in signed two's complement arithmetic, updates the destination register or memory location, and advances the PC. A separate `reg_written[]` boolean array records whether each register was ever written, so registers whose final value is zero still appear in the final state output. Simulation halts on a HALT instruction.

3.2 Pipeline hazard handling

3.2.1 RAW data hazards – no forwarding

In the no-forwarding case, a consumer instruction must wait until the producer instruction has completed its Write Back (WB) stage. The stall penalty depends on pipeline distance when the hazard is detected at ID.

Table 3.1: RAW stall penalties without forwarding

Producer stage at consumer ID	Distance	Stalls	Reason
EX	1	2	Producer needs MEM and WB
MEM	2	1	Producer needs WB
WB	3	0	Same-cycle read/write rule applies

3.2.2 RAW data hazards – with forwarding

With full forwarding, results from EX and MEM stage outputs are forwarded directly to the EX stage inputs, eliminating all RAW stalls except the load-use hazard.

Table 3.2: RAW stall penalties with forwarding

Hazard type	Stall cycles	Notes
EX-EX forwarding	0	Result forwarded from EX output
MEM-EX forwarding	0	Result forwarded from MEM output
Load-use hazard (LDW at distance 1)	1	LDW result not ready until after MEM

3.3 Branch handling

The simulator uses an always-not-taken branch predictor. The branch condition and target address are resolved at the end of the EX stage. If a branch is taken (BZ, BEQ, or JR),

the two instructions fetched speculatively from the wrong path are flushed (converted to bubbles), and the PC is redirected to the branch target. This results in a 2-cycle branch penalty for every taken branch or jump. Not-taken conditional branches incur zero penalty.

Two pipeline bubbles are inserted as actual pipeline entries, not merely a cycle charge, so they correctly prevent false distance-2 RAW hazards on the first instruction at the correct target.

3.4 Assumptions

- All instructions complete in exactly 5 cycles in the absence of hazards.
- No WAR or WAW hazards occur due to the in-order 5-stage pipeline with single-issue execution.
- Register write occurs in the first half of the clock cycle and register read occurs in the second half, allowing WB and ID to share a cycle without a stall (same-cycle read/write rule).
- Memory accesses always complete in a single cycle; no cache model is included.
- The PC is initialized to 0 and all general-purpose registers are initialized to 0.
- Programs do not access data outside the 4 kB memory range.
- JR uses R[rs] as a byte address directly with no left shift.
- BEQ and BZ branch targets are computed as $PC + 4 \cdot imm$, where PC is the byte address of the branch instruction and imm is the signed 16-bit offset from the instruction encoding.

4 Design methodology

4.1 Introduction

This chapter describes the simulation methodology used to develop and validate the MIPS-Lite simulator. The simulator is organized into six cooperating modules sharing a common instruction decode front-end, with distinct back-ends for functional simulation and pipeline timing.

4.2 Simulation environment

The simulator reads the 4 kB memory image file, initializes machine state, and executes instructions sequentially, updating machine state and collecting statistics. The simulation terminates upon encountering the HALT instruction.

The simulation environment consists of the following components.

- Memory image reader – Parses the hex memory image and loads it into the simulated memory array.
- Instruction decoder – Extracts opcode, register fields, and immediate values from 32-bit instruction words.
- Functional execution engine – Applies instruction semantics to update register and memory state.
- No-forwarding timing model – Tracks pipeline stages and inserts stalls for all RAW hazards.
- Forwarding timing model – Tracks pipeline stages, applies bypass paths, and inserts stalls only for load-use hazards.
- Statistics collector – Counts instruction types, hazards, stall cycles, and total clock cycles.

4.3 Memory image format

The memory image is a text file where each line represents one 32-bit word in big-endian hexadecimal format. Addresses start at 0 and increment by 4 per line. The program counter begins at address 0, so execution starts from the first line of the image.

4.4 Instruction decoding

The simulator decodes each instruction by examining the 6-bit opcode field in the most significant bits of the 32-bit word. Based on the opcode, the instruction format (R-type or I-type) is determined and the appropriate fields are extracted. The decoded instruction is dispatched to the functional and timing execution engines. The immediate field is stored as `int16_t`, giving automatic two's complement sign extension with no manual bit manipulation.

4.5 Hazard detection

Hazard detection is performed at the ID stage by comparing the source register identifiers of the current instruction against the destination register identifiers of the instructions in the EX and MEM stages. (The WB stage is exempt by the same-cycle read/write rule.) If a match is found and the producing instruction writes a non-zero register, a hazard is flagged and the appropriate stall count is determined based on the stage distance and whether forwarding is enabled.

4.6 Simulation results

The simulator produces the following output for a given memory image.

- Instruction frequency breakdown by type (arithmetic, logical, memory access, control transfer).
- Final machine state: PC value, changed register values, and changed memory locations.
- Stall analysis for both no-forwarding and forwarding scenarios.
- Total execution time in clock cycles for each scenario.
- Speedup achieved by forwarding.

5 Implementation of the MIPS-Lite simulator

5.1 Introduction

This chapter describes the implementation of the MIPS-Lite simulator. The simulator is organized into modular components corresponding to the trace reader, instruction decoder, functional simulator, pipeline timing model, and statistics reporting module.

5.2 Trace reader module

`trace_reader.c/.h` opens the memory image file and reads it line by line. Each 8-character hex line is converted to a 32-bit unsigned integer and stored in a flat `uint32_t` array of 1024 entries. The array index directly encodes the byte address divided by 4, making address-to-line lookup a single array access. Blank lines are skipped; malformed lines produce an error message to `stderr`.

5.3 Instruction decoder module

`decoder.c/.h` extracts the 6-bit opcode from bits 31–26, then decodes the register fields and immediate. R-type instructions carry Rs (bits 25–21), Rt (bits 20–16), and Rd (bits 15–11). I-type instructions carry Rs, Rt, and a 16-bit immediate in bits 15–0. The immediate is stored as `int16_t`, giving automatic two’s complement sign extension on any C11-compliant compiler.

5.4 Functional simulator module

`func_sim.c/.h` maintains the complete machine state: a 32-bit program counter, 32 general-purpose registers (R0 hardwired to zero and write-protected), and the memory array. A separate `reg_written[]` boolean array records whether each register was ever written, so registers whose final value is zero still appear in the final state output.

5.5 Pipeline timing simulator module

`timing_sim.c/.h` observes the instruction stream produced by the functional simulator and maintains a 5-entry pipeline window. For each instruction it computes the number of stall cycles required (RAW hazard analysis) and whether a taken branch triggers a pipeline flush. The same source file handles both no-forwarding and forwarding modes via a `timing_mode_t` flag; no code is duplicated.

5.5.1 Stall insertion

When a RAW hazard is detected, the consumer instruction is held in IF. Instructions in ID through WB continue to advance. NOP bubbles are inserted into the EX stage for each stall cycle.

5.5.2 Branch flush

When a taken branch or jump is detected, two pipeline bubbles are inserted and two flush cycles are charged. The bubbles are actual pipeline entries, not just a cycle charge, so they correctly prevent false distance-2 RAW hazards between the branch and the instruction at the correct target.

5.5.3 Lockstep execution model

The functional and timing simulators run in lockstep. On each iteration, the functional simulator fetches and executes one instruction, updating the real machine state. The timing simulator then observes the same instruction and its taken/not-taken status and charges the appropriate number of clock cycles. Stall cycles are modeled by incrementing the clock counter and advancing the pipeline window without fetching a new instruction.

5.6 Statistics module

`stats.c/.h` collects the following metrics during simulation.

- Count of each instruction type (arithmetic, logical, memory, control).
- Total number of data hazards detected (for both forwarding and no-forwarding scenarios).
- Number of stall cycles contributed by each hazard type.
- Number of branch flush events and associated flush cycles.
- Total clock cycles for no-forwarding and forwarding scenarios.

6 Simulation results and analysis

All results in this chapter are from the demo-day memory image (`final_proj_trace.txt`).

6.1 Instruction frequency breakdown

Table 6.1: Instruction frequency breakdown

Instruction type	Count	Percentage (%)
Arithmetic	375	41.2
Logical	61	6.7
Memory access	300	32.9
Control transfer	175	19.2
Total	911	100.0

6.2 Final machine state

Only registers and memory locations whose contents changed during execution are reported. All values are decimal.

6.2.1 Program counter

Table 6.2: Final program counter

Item	Value (decimal)
Program counter (PC)	112

6.2.2 General-purpose registers

Fifteen registers changed state during execution. All others retained their initial value of 0.

Table 6.3: Final register state (changed registers only)

Register	Value (decimal)
R11	1044
R12	1836
R13	2640
R14	25
R15	−188
R16	213
R17	29
R18	3440
R19	−1
R20	−2
R21	−1
R22	76
R23	3
R24	−1
R25	3

The presence of negative values (R15, R19, R20, R21, R24) confirms correct signed two’s complement arithmetic throughout execution.

6.2.3 Memory

Sixty memory addresses changed during execution, spanning byte addresses 2400–2636. The full listing appears in Appendix B. A condensed view of selected entries is shown below.

Table 6.4: Final memory state – selected entries (full table in Appendix B)

Address (decimal)	Contents (decimal)
2400	2
2404	4
2408	6
⋮	
2632	18
2636	29

6.3 Stall conditions and penalties

6.3.1 Pipeline timing model

The simulator assumes an always-not-taken branch predictor and a 5-stage in-order pipeline. All instructions complete in exactly 5 cycles in the absence of hazards, so WAR and WAW hazards do not arise. Two classes of event cause the pipeline to deviate from one-instruction-per-cycle throughput.

- RAW (Read After Write) data hazards. A consumer instruction in ID reads a register that a producer instruction has not yet written back. The stall penalty depends on the pipeline distance between producer and consumer, and on whether forwarding is available.
- Taken branches and jumps. The branch outcome is resolved at the end of the EX stage. Two wrong-path instructions have entered IF and ID by the time the branch resolves. These are flushed, costing 2 cycles per taken BZ, BEQ, or JR.

6.3.2 Same-cycle read/write rule

The specification states that when a register read and a register write of the same register occur in the same clock cycle, the write completes in the first half-cycle and the new value can be read in the second half-cycle. This means a producer in WB and a consumer in ID reading the same register in the same cycle do not produce a hazard. This eliminates the distance-3 stall case in no-forwarding mode.

6.3.3 No-forwarding stall rules

Table 6.5: No-forwarding RAW hazard stall penalties

Producer stage	Distance	Stalls	Reason
EX	1	2	Producer needs MEM and WB
MEM	2	1	Producer needs WB
WB	3	0	Same-cycle rule applies
Any taken branch	–	2 flush	2 wrong-path instructions flushed

6.3.4 Forwarding stall rules

With forwarding, results from the EX and MEM stage outputs are forwarded directly to the EX stage inputs, eliminating all RAW stalls except the load-use hazard.

Table 6.6: Forwarding RAW hazard stall penalties

Hazard type	Stalls	Note
LDW followed by dependent instruction at distance 1 (load-use)	1	Cannot forward from MEM to EX in time
All other RAW hazards	0	Forwarding resolves
Taken branch or JR	2 flush	Branch resolution at end of EX

6.4 Pipeline diagram – first five instructions

The first five instructions of the demo-day trace are independent ADDI instructions with no inter-instruction dependencies. They flow through the 5-stage pipeline without stalls, completing in 9 cycles, illustrating the ideal one-instruction-per-cycle throughput.

Table 6.7: Pipeline diagram – first 5 instructions (no stalls)

Instruction	1	2	3	4	5	6	7	8	9
ADDI R1, R0, 400	IF	ID	EX	ME	WB				
ADDI R2, R0, -7		IF	ID	EX	ME	WB			
ADDI R3, R0, 4			IF	ID	EX	ME	WB		
ADDI R4, R0, 0				IF	ID	EX	ME	WB	
ADDI R5, R0, 3					IF	ID	EX	ME	WB

Total clock cycles for first 5 instructions = 9 — Total arithmetic instructions = 5

6.5 Data hazard statistics

Table 6.8: Data hazard statistics

Metric	No forwarding	With forwarding
Total data hazards detected	307	60
Hazards fully resolved by forwarding	N/A	247
Hazards requiring stalls	307	60 (load-use only)
Total stall cycles from data hazards	554	60
Average stall cycles per hazard	1.805	1.000

6.6 No-forwarding timing

Table 6.9: No-forwarding timing results

Metric	Value
Instructions executed	911
Pipeline drain cycles	4
Total stall cycles	554
Total flush cycles	238
Total clock cycles	1707
Number of RAW data hazards	307
Average stall penalty per hazard	1.805 cycles
CPI	1.8738

Cycle accounting: $911 + 4 + 554 + 238 = 1707$ ✓

6.7 Forwarding timing

Table 6.10: Forwarding timing results

Metric	Value
Instructions executed	911
Pipeline drain cycles	4
Total stall cycles	60
Total flush cycles	238
Total clock cycles	1213
Unrecoverable hazards (load-use only)	60
CPI	1.3315

Cycle accounting: $911 + 4 + 60 + 238 = 1213$ ✓

All 60 forwarding stall cycles are load-use hazards. Because each load-use hazard produces exactly one stall cycle, the stall cycle count equals the unrecoverable hazard count. Flush cycles are identical between both modes (238) because forwarding has no effect on branch resolution timing.

6.8 Execution time and speedup

Table 6.11: Execution time and forwarding speedup

Metric	Value
Total clock cycles (no forwarding)	1707
Total clock cycles (with forwarding)	1213
Speedup (forwarding vs. no forwarding)	1.4073

$$\text{Speedup} = \frac{\text{Cycles (no forwarding)}}{\text{Cycles (forwarding)}} = \frac{1707}{1213} \approx 1.4073$$

Forwarding reduces total execution time by roughly 28.9 percent relative to the no-forwarding baseline. Of the 554 no-forwarding stall cycles, 494 are eliminated by forwarding. The remaining 60 are load-use hazards that forwarding cannot resolve.

6.9 Complete results summary

Table 6.12: Complete results summary

Metric	No forwarding	Forwarding
Total instructions	911	911
Arithmetic	375	375
Logical	61	61
Memory access	300	300
Control transfer	175	175
Total clock cycles	1707	1213
Stall cycles	554	60
Flush cycles	238	238
Data hazards detected	307	60
Avg stall penalty/hazard	1.805	1.000
CPI	1.874	1.332
Speedup	1.4073×	

6.10 Simulator output screenshots

The following figures show the complete terminal output of the simulator on the demo-day trace (`final_proj_trace.txt`).

```
Instruction counts:

Total number of instructions: 911
Arithmetic instructions: 375
Logical instructions: 61
Memory access instructions: 300
Control transfer instructions: 175

Final register state:

Program counter: 112
R11 : 1044
R12 : 1836
R13 : 2640
R14 : 25
R15 : -188
R16 : 213
R17 : 29
R18 : 3440
R19 : -1
R20 : -2
R21 : -1
R22 : 76
R23 : 3
R24 : -1
R25 : 3
```

Figure 6.1: Simulator output – instruction counts and final register state

Final memory state:

Address: 2400	, Contents: 2
Address: 2404	, Contents: 4
Address: 2408	, Contents: 6
Address: 2412	, Contents: 8
Address: 2416	, Contents: 10
Address: 2420	, Contents: 12
Address: 2424	, Contents: 14
Address: 2428	, Contents: 16
Address: 2432	, Contents: 18
Address: 2436	, Contents: 29
Address: 2440	, Contents: 22
Address: 2444	, Contents: 24
Address: 2448	, Contents: 26
Address: 2452	, Contents: 28
Address: 2456	, Contents: 30
Address: 2460	, Contents: 32
Address: 2464	, Contents: 34
Address: 2468	, Contents: 36
Address: 2472	, Contents: 38
Address: 2476	, Contents: 59
Address: 2480	, Contents: 42
Address: 2484	, Contents: 44
Address: 2488	, Contents: 46
Address: 2492	, Contents: 48
Address: 2496	, Contents: 50
Address: 2500	, Contents: 52
Address: 2504	, Contents: 54
Address: 2508	, Contents: 56
Address: 2512	, Contents: 58
Address: 2516	, Contents: 89
Address: 2520	, Contents: 62
Address: 2524	, Contents: 64
Address: 2528	, Contents: 66
Address: 2532	, Contents: 68
Address: 2536	, Contents: 70
Address: 2540	, Contents: 72
Address: 2544	, Contents: 74
Address: 2548	, Contents: 76
Address: 2552	, Contents: 78
Address: 2556	, Contents: 110

```
Address: 2604 ,Contents: 4
Address: 2608 ,Contents: 6
Address: 2612 ,Contents: 8
Address: 2616 ,Contents: 10
Address: 2620 ,Contents: 12
Address: 2624 ,Contents: 14
Address: 2628 ,Contents: 16
Address: 2632 ,Contents: 18
Address: 2636 ,Contents: 29

Timing Simulator (no forwarding):
Total clock cycles: 1707
Total stall cycles: 554
CPI: 1.8738
Total RAW data hazards: 307
Average stall penalty per hazard: 1.805 cycles
Flush cycles (taken branches/JR): 238

Timing Simulator (forwarding):
Total clock cycles: 1213
Total stall cycles: 60
CPI: 1.3315
RAW hazards not eliminated by forwarding: 60
Flush cycles (taken branches/JR): 238

Speedup of forwarding vs no-forwarding: 1.4073

Program Halted
```

Figure 6.3: Simulator output – timing results and speedup

7 Roles and responsibilities

All four team members contributed to the project. Work was distributed across design, implementation, testing, and documentation. The table below lists each member's primary modules and specific implementation responsibilities.

Table 7.1: Project roles and responsibilities (part 1 of 2)

Team member	Contributions
Nikhil Swarna	Trace reader, types, main driver, instruction decoder, statistics. Implemented <code>trace_reader.c/.h</code> : memory image file loading, line-by-line hex parsing, whitespace trimming, 8-digit validation, and bounds-checked word storage. Defined <code>types.h</code> : all shared enums (<code>opcode_t</code> , <code>format_t</code> , <code>category_t</code> , <code>stage_t</code>), <code>instruction_t</code> struct, and project-wide constants (<code>MEM_WORDS</code> , <code>NUM_REGS</code> , <code>PIPE_STAGES</code>). Implemented <code>decoder.c/.h</code> : opcode-indexed metadata table mapping all 18 opcodes to mnemonic, format, and category; <code>decoder_decode()</code> extracting rs, rt, rd, and sign-extended imm from the 32-bit word; and <code>decoder_mnemonic()</code> for readable opcode labels. Implemented <code>stats.c/.h</code> : counters for total, arithmetic, logical, memory, and control instructions; <code>stats_count_instruction()</code> dispatch; and <code>stats_print_instruction_counts()</code> report formatting.
Venkat Sai Sumanth Koyada	Timing simulator – no-forwarding mode (Mode B). Implemented <code>timing_sim.c/.h</code> for the no-forwarding pipeline: <code>timing_t</code> pipeline struct with 5-slot pipe array, clock/stall/hazard/flush counters, and mode flag; <code>dest_reg()</code> and <code>src_regs()</code> helpers identifying producer/consumer register dependencies; <code>produces_for()</code> RAW dependency check; <code>compute_stalls()</code> implementing no-forwarding stall rules (2 cycles at distance 1, 1 cycle at distance 2, 0 at distance 3); <code>shift()</code> pipeline advance; <code>timing_step()</code> driving fetch, stall insertion, and branch flush; <code>timing_drain()</code> draining the last 4 stages; and <code>timing_print_report()</code> formatting CPI, hazard counts, and flush cycles.

Table 7.2: Project roles and responsibilities (part 2 of 2)

Team member	Contributions
Venkata Sriram Kamarajugadda	Functional simulator and main driver. Implemented <code>func_sim.c/.h</code> : <code>machine_t</code> state struct (PC, 32-entry register file, <code>reg_written</code> change log, memory pointer); <code>func_sim_init()</code> zeroing all state; <code>func_sim_step()</code> executing all 18 instructions using signed 32-bit two's complement arithmetic; <code>reg_write()</code> helper enforcing R0 hard-wiring; <code>word_index()</code> helper for aligned bounds-checked memory access; effective address computation for LDW/STW; and PC update logic for BZ, BEQ, and JR. Implemented <code>main.c</code> : top-level simulation driver that initializes machine state, runs the fetch-decode-execute loop, drives both timing simulators in lockstep, and formats the final report.
Hanisha Produtur	Timing simulator – forwarding mode (Mode C). Implemented <code>timing_sim.c/.h</code> for the forwarding pipeline: <code>timing_t</code> pipeline struct with 5-slot pipe array, clock/stall/hazard/flush counters, and mode flag; <code>dest_reg()</code> and <code>src_regs()</code> helpers; <code>produces_for()</code> RAW dependency check; <code>compute_stalls()</code> implementing forwarding rules (load-use 1 stall, all other RAW hazards 0 stalls); <code>shift()</code> pipeline advance; <code>timing_step()</code> driving fetch, stall insertion, and branch flush; <code>timing_drain()</code> draining the last 4 stages; and <code>timing_print_report()</code> formatting CPI, hazard counts, and flush cycles.

All members participated in design reviews, integration testing, and demo preparation.

Conclusion

A fully functional MIPS-Lite execution-driven pipeline simulator was implemented in C (C11). The simulator correctly handles all 17 MIPS-Lite instructions, including signed arithmetic, negative immediates, negative effective addresses, and all three branch/jump types. It produces correct functional and timing output (clock cycles, stall counts, flush counts, CPI, and speedup) for both the no-forwarding and forwarding pipeline models.

On the demo-day trace the forwarding pipeline achieved a speedup of 1.4073 over the no-forwarding baseline. The dominant source of overhead in the no-forwarding case is distance-1 RAW hazards (2 cycles each). Forwarding eliminates all such stalls except load-use hazards, which require a structural one-cycle stall because the loaded value does not exist until after the MEM stage.

The simulator was validated against the sample memory image reference output and passed all 57 assertions in the automated test suite. All internal cycle-accounting checks are satisfied for both timing modes on both the sample and demo traces.

A Source code

The project source tree is organized as follows.

- include/types.h – shared constants, opcode enum, instruction struct
- include/trace_reader.h / src/trace_reader.c – memory image loader
- include/decoder.h / src/decoder.c – instruction decoder
- include/func_sim.h / src/func_sim.c – functional simulator
- include/timing_sim.h / src/timing_sim.c – timing simulator (Modes B and C)
- include/stats.h / src/stats.c – statistics
- src/main.c – top-level driver

Build and run:

```
cd mips-lite
make clean && make
./mips-lite <memory_image.txt>
```

A.1 include/types.h

Listing A.1: types.h – shared type definitions

```
1  /*****
2  * types.h
3  *
4  * Shared definitions for the MIPS-lite simulator.
5  * Every module includes this. Do not put logic here, only types and constants.
6  *****/
7  #ifndef TYPES_H
8  #define TYPES_H
9
10 #include <stdint.h>
11
12 /* ----- Project-wide constants ----- */
13
14 #define MEM_WORDS 1024 /* 4 kB memory image, 1024 words of 4 bytes [spec] */
15 #define NUM_REGS 32 /* R0..R31, R0 hardwired to 0 */
16 #define PIPE_STAGES 5 /* IF, ID, EX, MEM, WB */
17
18 /* ----- Opcodes (values fixed by the spec) ----- */
19
20 typedef enum {
21     OP_ADD = 0,
22     OP_ADDI = 1,
23     OP_SUB = 2,
24     OP_SUBI = 3,
25     OP_MUL = 4,
26     OP_MULI = 5,
27     OP_OR = 6,
28     OP_ORI = 7,
29     OP_AND = 8,
30     OP_ANDI = 9,
31     OP_XOR = 10,
32     OP_XORI = 11,
33     OP_LDW = 12,
34     OP_STW = 13,
35     OP_BZ = 14,
36     OP_BEQ = 15,
```

```

37     OP_JR   = 16,
38     OP_HALT = 17,
39     OP_NOP  = 63 /* not a real MIPS-lite opcode, used for pipeline stalls */
40 } opcode_t;
41
42 /* ----- Instruction format ----- */
43
44 typedef enum {
45     FMT_R,
46     FMT_I,
47     FMT_INVALID
48 } format_t;
49
50 /* ----- Instruction category (for frequency report) ----- */
51
52 typedef enum {
53     CAT_ARITHMETIC,
54     CAT_LOGICAL,
55     CAT_MEMORY,
56     CAT_CONTROL,
57     CAT_INVALID
58 } category_t;
59
60 /* ----- Pipeline stage indices ----- */
61
62 typedef enum {
63     IF = 0,
64     ID = 1,
65     EX = 2,
66     MEM = 3,
67     WB = 4
68 } stage_t;
69
70 /* ----- Decoded instruction ----- */
71
72 typedef struct {
73     uint32_t raw; /* original 32-bit word, kept for debugging */
74     opcode_t opcode;
75     format_t format;
76     category_t category;
77     uint8_t rs; /* 5-bit source register */
78     uint8_t rt; /* 5-bit second source / I-type destination */
79     uint8_t rd; /* 5-bit R-type destination (0 if I-type) */
80     int16_t imm; /* signed 16-bit immediate (I-type only) */
81 } instruction_t;
82
83 #endif /* TYPES_H */

```

A.2 include/decoder.h

Listing A.2: decoder.h

```

1  /*****
2  * decoder.h
3  *
4  * Decodes one 32-bit MIPS-lite word into an instruction_t struct.
5  *****/
6  #ifndef DECODER_H
7  #define DECODER_H
8
9  #include <stdint.h>
10 #include "types.h"
11
12 /*
13  * Decode a single 32-bit word into the instruction struct pointed to by 'out'.

```

```

14  * Always fills every field. For R-type, imm is 0. For I-type, rd is 0.
15  *
16  * Returns 0 on success, nonzero if the opcode is not recognized.
17  */
18  int decoder_decode(uint32_t word, instruction_t *out);
19
20  /* Returns a short uppercase mnemonic for the opcode, e.g. "ADDI", "HALT". */
21  const char *decoder_mnemonic(opcode_t op);
22
23  #endif /* DECODER_H */

```

A.3 src/decoder.c

Listing A.3: decoder.c – instruction decoder

```

1  /*****
2  * decoder.c
3  *
4  * Decodes one 32-bit MIPS-lite word. Uses a static table to map opcodes to
5  * their format (R/I) and category (arith/logic/mem/ctrl).
6  *****/
7  #include "decoder.h"
8  #include <stdio.h>
9
10 typedef struct {
11     const char *mnemonic;
12     format_t    format;
13     category_t  category;
14 } opinfo_t;
15
16 static const opinfo_t OPINFO[] = {
17     [OP_ADD] = {"ADD", FMT_R, CAT_ARITHMETIC},
18     [OP_ADDI] = {"ADDI", FMT_I, CAT_ARITHMETIC},
19     [OP_SUB] = {"SUB", FMT_R, CAT_ARITHMETIC},
20     [OP_SUBI] = {"SUBI", FMT_I, CAT_ARITHMETIC},
21     [OP_MUL] = {"MUL", FMT_R, CAT_ARITHMETIC},
22     [OP_MULI] = {"MULI", FMT_I, CAT_ARITHMETIC},
23     [OP_OR] = {"OR", FMT_R, CAT_LOGICAL},
24     [OP_ORI] = {"ORI", FMT_I, CAT_LOGICAL},
25     [OP_AND] = {"AND", FMT_R, CAT_LOGICAL},
26     [OP_ANDI] = {"ANDI", FMT_I, CAT_LOGICAL},
27     [OP_XOR] = {"XOR", FMT_R, CAT_LOGICAL},
28     [OP_XORI] = {"XORI", FMT_I, CAT_LOGICAL},
29     [OP_LDW] = {"LDW", FMT_I, CAT_MEMORY},
30     [OP_STW] = {"STW", FMT_I, CAT_MEMORY},
31     [OP_BZ] = {"BZ", FMT_I, CAT_CONTROL},
32     [OP_BEQ] = {"BEQ", FMT_I, CAT_CONTROL},
33     [OP_JR] = {"JR", FMT_I, CAT_CONTROL},
34     [OP_HALT] = {"HALT", FMT_I, CAT_CONTROL},
35 };
36
37 #define OPINFO_COUNT (sizeof(OPINFO) / sizeof(OPINFO[0]))
38
39 int decoder_decode(uint32_t word, instruction_t *out) {
40     uint8_t op = (word >> 26) & 0x3F;
41
42     if (op >= OPINFO_COUNT || OPINFO[op].mnemonic == NULL) {
43         fprintf(stderr, "decoder: unknown opcode %u in word 0x%08X\n",
44             op, word);
45         out->opcode = OP_NOP;
46         out->format = FMT_INVALID;
47         out->category = CAT_INVALID;
48         return 1;
49     }
50 }

```

```

51     out->raw      = word;
52     out->opcode   = (opcode_t)op;
53     out->format   = OPINFO[op].format;
54     out->category = OPINFO[op].category;
55
56     out->rs = (word >> 21) & 0x1F;
57     out->rt = (word >> 16) & 0x1F;
58
59     if (out->format == FMT_R) {
60         out->rd = (word >> 11) & 0x1F;
61         out->imm = 0;
62     } else {
63         out->rd = 0;
64         out->imm = (int16_t)(word & 0xFFFF);
65     }
66     return 0;
67 }
68
69 const char *decoder_mnemonic(opcode_t op) {
70     if ((unsigned)op == OP_NOP)
71         return "NOP";
72     if ((unsigned)op >= OPINFO_COUNT || OPINFO[op].mnemonic == NULL)
73         return "???";
74     return OPINFO[op].mnemonic;
75 }

```

A.4 include/func_sim.h

Listing A.4: func_sim.h

```

1  /*****
2  * func_sim.h
3  *
4  * Functional simulator: executes one decoded instruction against machine
5  * state (PC, registers, memory).
6  *****/
7  #ifndef FUNC_SIM_H
8  #define FUNC_SIM_H
9
10 #include <stdint.h>
11 #include <stdbool.h>
12 #include "types.h"
13
14 /* Machine state. PC is a byte address. regs[0] is always 0. */
15 typedef struct {
16     uint32_t pc;
17     int32_t  regs[NUM_REGS];
18     bool     reg_written[NUM_REGS]; /* true if this reg was ever written */
19     uint32_t *mem;
20     int      mem_words;
21     bool     halted;
22 } machine_t;
23
24 /* Initialize the machine. Does not own 'mem'; the caller does. */
25 void func_sim_init(machine_t *m, uint32_t *mem, int mem_words);
26
27 /* Execute one instruction (already decoded). Updates PC, regs, mem as needed.
28  * Returns 0 on success, nonzero on error (e.g., bad memory access). */
29 int func_sim_step(machine_t *m, const instruction_t *ins);
30
31 #endif /* FUNC_SIM_H */

```

A.5 src/func_sim.c

Listing A.5: func_sim.c – functional simulator

```

1  /*****
2  * func_sim.c
3  *
4  * Executes one MIPS-lite instruction against machine state.
5  * Arithmetic is signed two's complement. R0 is hardwired to 0.
6  *****/
7  #include "func_sim.h"
8  #include "decoder.h"
9  #include <stdio.h>
10
11 static void reg_write(machine_t *m, uint8_t r, int32_t v) {
12     if (r != 0) {
13         m->regs[r] = v;
14         m->reg_written[r] = true;
15     }
16 }
17
18 static int word_index(const machine_t *m, uint32_t byte_addr) {
19     if ((byte_addr & 0x3) != 0) {
20         fprintf(stderr, "func_sim: unaligned address 0x%08X\n", byte_addr);
21         return -1;
22     }
23     int idx = (int)(byte_addr / 4);
24     if (idx < 0 || idx >= m->mem_words) {
25         fprintf(stderr, "func_sim: address 0x%08X out of range\n", byte_addr);
26         return -1;
27     }
28     return idx;
29 }
30
31 void func_sim_init(machine_t *m, uint32_t *mem, int mem_words) {
32     m->pc = 0;
33     for (int i = 0; i < NUM_REGS; i++) {
34         m->regs[i] = 0;
35         m->reg_written[i] = false;
36     }
37     m->mem = mem;
38     m->mem_words = mem_words;
39     m->halted = false;
40 }
41
42 int func_sim_step(machine_t *m, const instruction_t *ins) {
43     uint32_t next_pc = m->pc + 4;
44
45     int32_t rs_val = m->regs[ins->rs];
46     int32_t rt_val = m->regs[ins->rt];
47     int32_t imm = (int32_t)ins->imm;
48
49     uint32_t a = (uint32_t)rs_val;
50     uint32_t b = (uint32_t)rt_val;
51     uint32_t i = (uint32_t)imm;
52
53     switch (ins->opcode) {
54     case OP_ADD: reg_write(m, ins->rd, (int32_t)(a + b)); break;
55     case OP_ADDI: reg_write(m, ins->rt, (int32_t)(a + i)); break;
56     case OP_SUB: reg_write(m, ins->rd, (int32_t)(a - b)); break;
57     case OP_SUBI: reg_write(m, ins->rt, (int32_t)(a - i)); break;
58     case OP_MUL: reg_write(m, ins->rd, (int32_t)(a * b)); break;
59     case OP_MULI: reg_write(m, ins->rt, (int32_t)(a * i)); break;
60     case OP_OR: reg_write(m, ins->rd, (int32_t)(a | b)); break;
61     case OP_ORI: reg_write(m, ins->rt, (int32_t)(a | i)); break;
62     case OP_AND: reg_write(m, ins->rd, (int32_t)(a & b)); break;
63     case OP_ANDI: reg_write(m, ins->rt, (int32_t)(a & i)); break;
64     case OP_XOR: reg_write(m, ins->rd, (int32_t)(a ^ b)); break;

```

```

65     case OP_XORI: reg_write(m, ins->rt, (int32_t)(a ^ i)); break;
66     case OP_LDW: {
67         uint32_t addr = (uint32_t)(rs_val + imm);
68         int idx = word_index(m, addr);
69         if (idx < 0) return 1;
70         reg_write(m, ins->rt, (int32_t)m->mem[idx]);
71         break;
72     }
73     case OP_STW: {
74         uint32_t addr = (uint32_t)(rs_val + imm);
75         int idx = word_index(m, addr);
76         if (idx < 0) return 1;
77         m->mem[idx] = (uint32_t)rt_val;
78         break;
79     }
80     case OP_BZ:
81         if (rs_val == 0)
82             next_pc = m->pc + (uint32_t)(imm * 4);
83         break;
84     case OP_BEQ:
85         if (rs_val == rt_val)
86             next_pc = m->pc + (uint32_t)(imm * 4);
87         break;
88     case OP_JR:
89         next_pc = (uint32_t)rs_val;
90         break;
91     case OP_HALT:
92         m->halted = true;
93         break;
94     default:
95         fprintf(stderr, "func_sim: unhandled opcode %s\n",
96                 decoder_mnemonic(ins->opcode));
97         return 1;
98     }
99
100     m->pc = next_pc;
101     return 0;
102 }

```

A.6 include/timing_sim.h

Listing A.6: timing_sim.h

```

1  /*****
2  * timing_sim.h
3  *
4  * Pipeline timing model. Counts clock cycles, RAW stalls, and branch flushes.
5  * Two modes: no forwarding, and forwarding.
6  *****/
7  #ifndef TIMING_SIM_H
8  #define TIMING_SIM_H
9
10 #include <stdbool.h>
11 #include "types.h"
12
13 typedef enum {
14     MODE_NO_FORWARDING,
15     MODE_FORWARDING
16 } timing_mode_t;
17
18 typedef struct {
19     timing_mode_t mode;
20     long          clock;
21     long          stall_cycles;
22     long          hazard_count;

```

```

23     long        flush_cycles;
24     instruction_t pipe[PIPE_STAGES];
25 } timing_t;
26
27 void timing_init(timing_t *t, timing_mode_t mode);
28
29 /* Feed one program-order instruction. taken_branch is true if this
30  * instruction was a taken BZ/BEQ or any JR. */
31 void timing_step(timing_t *t, const instruction_t *ins, bool taken_branch);
32
33 /* After the last instruction, drain the remaining 4 stages. */
34 void timing_drain(timing_t *t);
35
36 /* Print the timing section of the report. */
37 void timing_print_report(const timing_t *t, long instr_count);
38
39 #endif /* TIMING_SIM_H */

```

A.7 src/timing_sim.c

Listing A.7: timing_sim.c – timing simulator

```

1  /*****
2  * timing_sim.c
3  *
4  * Pipeline timing model. Tracks the 5-stage pipeline, counts RAW stall
5  * cycles and branch flush cycles.
6  *
7  * Pipeline indices: IF=0, ID=1, EX=2, MEM=3, WB=4.
8  * New instructions enter at IF (index 0). Each cycle, everything shifts up
9  * by one index. WB (index 4) retires.
10  *****/
11 #include "timing_sim.h"
12 #include "decoder.h"
13 #include <stdio.h>
14
15 static const instruction_t BUBBLE = {
16     .raw = 0, .opcode = OP_NOP, .format = FMT_INVALID,
17     .category = CAT_INVALID,
18     .rs = 0, .rt = 0, .rd = 0, .imm = 0
19 };
20
21 static int dest_reg(const instruction_t *ins) {
22     switch (ins->opcode) {
23         case OP_ADD: case OP_SUB: case OP_MUL:
24         case OP_OR: case OP_AND: case OP_XOR:
25             return ins->rd;
26         case OP_ADDI: case OP_SUBI: case OP_MULI:
27         case OP_ORI: case OP_ANDI: case OP_XORI:
28         case OP_LDW:
29             return ins->rt;
30         default:
31             return -1;
32     }
33 }
34
35 static void src_regs(const instruction_t *ins, int *src1, int *src2) {
36     *src1 = -1; *src2 = -1;
37     switch (ins->opcode) {
38         case OP_ADD: case OP_SUB: case OP_MUL:
39         case OP_OR: case OP_AND: case OP_XOR:
40             *src1 = ins->rs; *src2 = ins->rt; break;
41         case OP_ADDI: case OP_SUBI: case OP_MULI:
42         case OP_ORI: case OP_ANDI: case OP_XORI:
43         case OP_LDW:

```

```

44     *src1 = ins->rs; break;
45     case OP_STW:
46         *src1 = ins->rs; *src2 = ins->rt; break;
47     case OP_BZ:    *src1 = ins->rs; break;
48     case OP_BEQ:   *src1 = ins->rs; *src2 = ins->rt; break;
49     case OP_JR:    *src1 = ins->rs; break;
50     default: break;
51 }
52 }
53
54 static int produces_for(const instruction_t *producer,
55                        const instruction_t *consumer) {
56     int dst = dest_reg(producer);
57     if (dst <= 0) return 0;
58     int s1, s2;
59     src_regs(consumer, &s1, &s2);
60     return (dst == s1) || (dst == s2);
61 }
62
63 static int compute_stalls(const timing_t *t, const instruction_t *consumer) {
64     const instruction_t *p_dist1 = &t->pipe[ID];
65     const instruction_t *p_dist2 = &t->pipe[EX];
66
67     if (t->mode == MODE_NO_FORWARDING) {
68         if (produces_for(p_dist1, consumer)) return 2;
69         if (produces_for(p_dist2, consumer)) return 1;
70         return 0;
71     }
72     /* Forwarding: load-use at distance 1 stalls 1 cycle, else 0 */
73     if (p_dist1->opcode == OP_LDW && produces_for(p_dist1, consumer))
74         return 1;
75     return 0;
76 }
77
78 static void shift(timing_t *t, instruction_t new_if) {
79     t->pipe[WB] = t->pipe[MEM];
80     t->pipe[MEM] = t->pipe[EX];
81     t->pipe[EX] = t->pipe[ID];
82     t->pipe[ID] = t->pipe[IF];
83     t->pipe[IF] = new_if;
84 }
85
86 void timing_init(timing_t *t, timing_mode_t mode) {
87     t->mode = mode;
88     t->clock = 0;
89     t->stall_cycles = 0;
90     t->hazard_count = 0;
91     t->flush_cycles = 0;
92     for (int i = 0; i < PIPE_STAGES; i++)
93         t->pipe[i] = BUBBLE;
94 }
95
96 void timing_step(timing_t *t, const instruction_t *ins, bool taken_branch) {
97     shift(t, *ins);
98     t->clock++;
99
100     int stalls = compute_stalls(t, &t->pipe[IF]);
101     if (stalls > 0) {
102         t->hazard_count++;
103         t->stall_cycles += stalls;
104         for (int i = 0; i < stalls; i++) {
105             t->pipe[WB] = t->pipe[MEM];
106             t->pipe[MEM] = t->pipe[EX];
107             t->pipe[EX] = t->pipe[ID];
108             t->pipe[ID] = BUBBLE;
109             t->clock++;
110         }
111     }

```

```

112
113     if (taken_branch) {
114         for (int i = 0; i < 2; i++) {
115             shift(t, BUBBLE);
116             t->clock++;
117         }
118         t->flush_cycles += 2;
119     }
120 }
121
122 void timing_drain(timing_t *t) {
123     for (int i = 0; i < 4; i++) {
124         shift(t, BUBBLE);
125         t->clock++;
126     }
127 }
128
129 void timing_print_report(const timing_t *t, long instr_count) {
130     const char *label = (t->mode == MODE_NO_FORWARDING)
131         ? "no forwarding" : "forwarding";
132     printf("\nTiming Simulator (%s):\n", label);
133     printf("Total clock cycles: %ld\n", t->clock);
134     printf("Total stall cycles: %ld\n", t->stall_cycles);
135     double cpi = (instr_count > 0)
136         ? (double)t->clock / (double)instr_count : 0.0;
137     printf("CPI: %.4f\n", cpi);
138
139     if (t->mode == MODE_NO_FORWARDING) {
140         printf("Total RAW data hazards: %ld\n", t->hazard_count);
141         double avg = (t->hazard_count > 0)
142             ? (double)t->stall_cycles / (double)t->hazard_count
143             : 0.0;
144         printf("Average stall penalty per hazard: %.3f cycles\n", avg);
145     } else {
146         printf("RAW hazards not eliminated by forwarding: %ld\n",
147             t->hazard_count);
148     }
149     printf("Flush cycles (taken branches/JR): %ld\n", t->flush_cycles);
150 }

```

A.8 include/stats.h

Listing A.8: stats.h

```

1  /*****
2  * stats.h
3  *
4  * Counters for the report: instruction-type frequencies.
5  *****/
6  #ifndef STATS_H
7  #define STATS_H
8
9  #include "types.h"
10
11  typedef struct {
12     int total;
13     int arithmetic;
14     int logical;
15     int memory;
16     int control;
17 } stats_t;
18
19 void stats_init(stats_t *s);
20 void stats_count_instruction(stats_t *s, const instruction_t *ins);
21 void stats_print_instruction_counts(const stats_t *s);

```

```

22
23 #endif /* STATS_H */

```

A.9 src/stats.c

Listing A.9: stats.c – statistics

```

1  /*****
2  * stats.c
3  *
4  * Accumulates per-category instruction counts and prints them.
5  *****/
6  #include "stats.h"
7  #include <stdio.h>
8
9  void stats_init(stats_t *s) {
10     s->total = s->arithmetic = s->logical = s->memory = s->control = 0;
11 }
12
13 void stats_count_instruction(stats_t *s, const instruction_t *ins) {
14     s->total++;
15     switch (ins->category) {
16     case CAT_ARITHMETIC: s->arithmetic++; break;
17     case CAT_LOGICAL:   s->logical++;   break;
18     case CAT_MEMORY:    s->memory++;    break;
19     case CAT_CONTROL:   s->control++;   break;
20     case CAT_INVALID:   break;
21     }
22 }
23
24 void stats_print_instruction_counts(const stats_t *s) {
25     printf("Instruction counts:\n\n");
26     printf("Total number of instructions: %d\n", s->total);
27     printf("Arithmetic instructions: %d\n",      s->arithmetic);
28     printf("Logical instructions: %d\n",         s->logical);
29     printf("Memory access instructions: %d\n",   s->memory);
30     printf("Control transfer instructions: %d\n", s->control);
31 }

```

A.10 include/trace_reader.h

Listing A.10: trace_reader.h

```

1  /*****
2  * trace_reader.h
3  *
4  * Reads a memory image text file (one 8-hex-digit word per line) into a
5  * uint32_t array. No decoding happens here.
6  *****/
7  #ifndef TRACE_READER_H
8  #define TRACE_READER_H
9
10 #include <stdint.h>
11
12 int trace_reader_load(const char *path, uint32_t *mem, int *out_words);
13
14 #endif /* TRACE_READER_H */

```

A.11 src/trace_reader.c

Listing A.11: trace_reader.c – memory image loader

```

1  /*****
2  * trace_reader.c
3  *
4  * Reads an 8-hex-digit-per-line memory image into a uint32_t array.
5  * Validates each line, errors out on malformed input.
6  *****/
7  #include "trace_reader.h"
8  #include "types.h"
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12 #include <ctype.h>
13 #include <errno.h>
14
15 static char *trim(char *s) {
16     while (*s && isspace((unsigned char)*s)) s++;
17     char *end = s + strlen(s);
18     while (end > s && isspace((unsigned char)end[-1])) end--;
19     *end = '\0';
20     return s;
21 }
22
23 static int is_valid_hex_word(const char *s) {
24     int count = 0;
25     for (const char *p = s; *p; p++) {
26         if (!isxdigit((unsigned char)*p)) return 0;
27         count++;
28     }
29     return count == 8;
30 }
31
32 int trace_reader_load(const char *path, uint32_t *mem, int *out_words) {
33     FILE *fp = fopen(path, "r");
34     if (!fp) {
35         fprintf(stderr, "trace_reader: cannot open '%s': %s\n",
36             path, strerror(errno));
37         return 1;
38     }
39
40     char line[64];
41     int count = 0;
42     int lineno = 0;
43
44     while (fgets(line, sizeof line, fp) != NULL) {
45         lineno++;
46         char *trimmed = trim(line);
47         if (*trimmed == '\0') continue;
48
49         if (!is_valid_hex_word(trimmed)) {
50             fprintf(stderr,
51                 "trace_reader: line %d: expected 8 hex digits, got '%s'\n",
52                 lineno, trimmed);
53             fclose(fp);
54             return 2;
55         }
56         if (count >= MEM_WORDS) {
57             fprintf(stderr,
58                 "trace_reader: more than %d words in file (spec limit)\n",
59                 MEM_WORDS);
60             fclose(fp);
61             return 3;
62         }
63         errno = 0;
64         unsigned long val = strtoul(trimmed, NULL, 16);
65         if (errno != 0) {
66             fprintf(stderr, "trace_reader: line %d: parse error\n", lineno);

```

```

67         fclose(fp);
68         return 4;
69     }
70     mem[count++] = (uint32_t)val;
71 }
72
73 fclose(fp);
74 *out_words = count;
75 return 0;
76 }

```

A.12 src/main.c

Listing A.12: main.c – top-level driver

```

1  /*****
2  * main.c
3  *
4  * Functional + timing simulator driver. Runs the program once through the
5  * functional simulator, drives two timing simulators (no-forwarding and
6  * forwarding) in lockstep, then prints the full report including speedup.
7  *****/
8  #include <stdio.h>
9  #include <string.h>
10 #include "types.h"
11 #include "trace_reader.h"
12 #include "decoder.h"
13 #include "func_sim.h"
14 #include "stats.h"
15 #include "timing_sim.h"
16
17 int main(int argc, char **argv) {
18     if (argc != 2) {
19         fprintf(stderr, "usage: %s <memory_image.txt>\n", argv[0]);
20         return 1;
21     }
22
23     uint32_t mem[MEM_WORDS] = {0};
24     int words = 0;
25     if (trace_reader_load(argv[1], mem, &words) != 0)
26         return 1;
27
28     uint32_t mem_init[MEM_WORDS];
29     for (int i = 0; i < MEM_WORDS; i++)
30         mem_init[i] = mem[i];
31
32     machine_t m;
33     func_sim_init(&m, mem, words);
34
35     stats_t stats;
36     stats_init(&stats);
37
38     timing_t t_nofwd, t_fwd;
39     timing_init(&t_nofwd, MODE_NO_FORWARDING);
40     timing_init(&t_fwd, MODE_FORWARDING);
41
42     int instr_count = 0;
43     while (!m.halted) {
44         int pc_idx = (int)(m.pc / 4);
45         if (pc_idx < 0 || pc_idx >= words) {
46             fprintf(stderr, "main: PC=0x%08X out of range\n", m.pc);
47             return 1;
48         }
49         instruction_t ins;
50         if (decoder_decode(mem[pc_idx], &ins) != 0) {

```

```

51     fprintf(stderr, "main: decode failed at PC=0x%08X\n", m.pc);
52     return 1;
53 }
54
55 uint32_t pc_before = m.pc;
56 if (func_sim_step(&m, &ins) != 0)
57     return 1;
58 bool taken = (m.pc != pc_before + 4);
59
60 stats_count_instruction(&stats, &ins);
61 timing_step(&t_nofwd, &ins, taken);
62 timing_step(&t_fwd, &ins, taken);
63
64 instr_count++;
65 if (instr_count > 10000000) {
66     fprintf(stderr, "main: instruction count exceeded 10M\n");
67     return 1;
68 }
69 }
70
71 timing_drain(&t_nofwd);
72 timing_drain(&t_fwd);
73
74 stats_print_instruction_counts(&stats);
75
76 printf("\nFinal register state:\n\n");
77 printf("Program counter: %u\n", m.pc);
78 for (int r = 1; r < NUM_REGS; r++) {
79     if (m.reg_written[r])
80         printf("R%-2d : %d\n", r, m.regs[r]);
81 }
82
83 printf("\nFinal memory state:\n");
84 for (int i = 0; i < words; i++) {
85     if (mem[i] != mem_init[i])
86         printf("Address: %d ,Contents: %d\n", i * 4, (int32_t)mem[i]);
87 }
88
89 timing_print_report(&t_nofwd, instr_count);
90 timing_print_report(&t_fwd, instr_count);
91
92 double speedup = (t_fwd.clock > 0)
93     ? (double)t_nofwd.clock / (double)t_fwd.clock : 0.0;
94 printf("\nSpeedup of forwarding vs no-forwarding: %.4f\n", speedup);
95 printf("\nProgram Halted\n");
96 return 0;
97 }

```

B Sample memory image reference output

The following is the complete simulator output for the sample memory image (ECE586_sample_memory_image.txt) provided with the project specification. This output was used to validate the simulator before the demo day.

Listing B.1: Simulator output – sample memory image

```
Instruction counts:

Total number of instructions: 638
Arithmetic instructions: 333
Logical instructions: 50
Memory access instructions: 103
Control transfer instructions: 152

Final register state:

Program counter: 100
R1 : 1200
R2 : 1400
R3 : 100
R4 : 50
R5 : 50
R6 : 0
R7 : 25
R8 : 2550
R9 : 1275
R10 : 50
R11 : 50
R12 : 32

Final memory state:

Address: 1400 ,Contents: 25
Address: 1404 ,Contents: 2550
Address: 1408 ,Contents: 1275

Timing Simulator (no forwarding):
Total clock cycles: 1095
Total stall cycles: 301
CPI: 1.7163
Total RAW data hazards: 151
Average stall penalty per hazard: 1.993 cycles
Flush cycles (taken branches/JR): 152

Timing Simulator (forwarding):
Total clock cycles: 844
Total stall cycles: 50
CPI: 1.3230
RAW hazards not eliminated by forwarding: 50
Flush cycles (taken branches/JR): 152

Speedup of forwarding vs no-forwarding: 1.2974

Program Halted
```

C Demo-day trace – complete memory output

Complete final memory state for the demo-day trace (`final_proj_trace.txt`). All values are decimal. 60 addresses changed during execution, spanning byte addresses 2400 to 2636.

Listing C.1: Demo-day trace – complete final memory state

```
Final memory state:

Address: 2400 ,Contents: 2
Address: 2404 ,Contents: 4
Address: 2408 ,Contents: 6
Address: 2412 ,Contents: 8
Address: 2416 ,Contents: 10
Address: 2420 ,Contents: 12
Address: 2424 ,Contents: 14
Address: 2428 ,Contents: 16
Address: 2432 ,Contents: 18
Address: 2436 ,Contents: 29
Address: 2440 ,Contents: 22
Address: 2444 ,Contents: 24
Address: 2448 ,Contents: 26
Address: 2452 ,Contents: 28
Address: 2456 ,Contents: 30
Address: 2460 ,Contents: 32
Address: 2464 ,Contents: 34
Address: 2468 ,Contents: 36
Address: 2472 ,Contents: 38
Address: 2476 ,Contents: 59
Address: 2480 ,Contents: 42
Address: 2484 ,Contents: 44
Address: 2488 ,Contents: 46
Address: 2492 ,Contents: 48
Address: 2496 ,Contents: 50
Address: 2500 ,Contents: 52
Address: 2504 ,Contents: 54
Address: 2508 ,Contents: 56
Address: 2512 ,Contents: 58
Address: 2516 ,Contents: 89
Address: 2520 ,Contents: 62
Address: 2524 ,Contents: 64
Address: 2528 ,Contents: 66
Address: 2532 ,Contents: 68
Address: 2536 ,Contents: 70
Address: 2540 ,Contents: 72
Address: 2544 ,Contents: 74
Address: 2548 ,Contents: 76
Address: 2552 ,Contents: 78
Address: 2556 ,Contents: 119
Address: 2560 ,Contents: 82
Address: 2564 ,Contents: 84
Address: 2568 ,Contents: 86
Address: 2572 ,Contents: 88
Address: 2576 ,Contents: 90
Address: 2580 ,Contents: 92
Address: 2584 ,Contents: 94
Address: 2588 ,Contents: 96
Address: 2592 ,Contents: 98
Address: 2596 ,Contents: 149
Address: 2600 ,Contents: 2
Address: 2604 ,Contents: 4
Address: 2608 ,Contents: 6
Address: 2612 ,Contents: 8
```

Address: 2616 ,Contents: 10
Address: 2620 ,Contents: 12
Address: 2624 ,Contents: 14
Address: 2628 ,Contents: 16
Address: 2632 ,Contents: 18
Address: 2636 ,Contents: 29